

JavaScript Concepts Guide

Complete Reference with Syntax & Examples

MERN Stack Bootcamp - Tecstem Technologies

Generated: August 10, 2026

Table of Contents

- 1. Variables
- 2. Data Types
- 3. String Concatenation & Template Literals
- 4. Input/Output
- 5. Conditionals (if-else)
- 6. Nested Conditions
- 7. Functions
- 8. Parameters & Arguments
- 9. Return Statement
- 10. First-Class Functions
- 11. Callbacks
- 12. Arrow Functions
- 13. Scopes (Global & Local)
- 14. Lexical Scoping
- 15. Closures
- 16. Arrays
- 17. Array Methods
- 18. Loops
- 19. Objects
- 20. Destructuring
- 21. Typed.js Library
- 22. DOM

1. Variables

Definition: Named containers that store data values

Syntax:

```
let variableName = value;  
const variableName = value;  
var variableName = value;
```

Example:

```
let name = "Ahmed";  
const age = 20;  
let course = "MERN";
```

Key Points:

- Use **let** for block-scoped variables (recommended)
- Use **const** for constants (can't be reassigned)
- Avoid **var** (outdated, function-scoped)

2. Data Types

Definition: Different kinds of data: string, number, boolean, object, array, null, undefined

Examples:

```
// String  
let text = "Hello World";  
  
// Number  
let age = 20;  
let price = 99.99;  
  
// Boolean  
let isActive = true;  
  
// Array  
let names = ["Ahmed", "Umer", "Faizan"];  
  
// Object  
let student = { name: "Ahmed", age: 20 };
```

3. String Concatenation & Template Literals

Definition: Combining strings or embedding variables in strings

Method 1: Concatenation (+)

```
let message = "Hi, " + name + " you are " + age + " years old";
```

Method 2: Template Literals (backticks)

```
let message = `Hi, ${name} you are ${age} years old`;
```

Key Points:

- Template literals are cleaner and recommended
- Use backticks (`) not quotes
- Variables in \${} are replaced with their values

4. Input & Output

Definition: Getting data from user (input) and showing results (output)

Getting Input (prompt):

```
let name = prompt("Enter your name:");
```

```
let age = parseInt(prompt("Enter your age:"));
```

Showing Output (console.log):

```
console.log("Hello " + name);
```

```
console.log(`Your age is ${age}`);
```

Key Points:

- prompt() returns a string
- Use parseInt() to convert string to number
- console.log() displays in browser console

5. Conditionals (if-else)

Definition: Make decisions based on conditions

Syntax:

```
if (condition) {  
  // code if true  
} else if (anotherCondition) {  
  // code if first false, this true  
} else {  
  // code if all false  
}
```

Example:

```
if (age >= 18) {  
  console.log("You are an adult");  
} else {  
  console.log("You are a minor");  
}
```

6. Nested Conditions

Definition: if-else statements inside other if-else statements

Example:

```
if (marks >= 80) {  
  if (marks >= 90) {  
    console.log("Grade: A+");  
  } else {  
    console.log("Grade: A");  
  }  
}
```

Key Points:

- Check outer condition first
- Then check inner conditions
- Very useful for complex logic

7. Functions

Definition: Reusable blocks of code that perform specific tasks

Syntax:

```
function functionName() {  
  // code here  
}  
functionName(); // calling the function
```

Example:

```
function greet() {  
  console.log("Hello, User!");  
}  
greet(); // Output: Hello, User!
```

Key Points:

- Define function once, use many times
- Makes code organized and reusable
- Function name should describe what it does

8. Parameters & Arguments

Definition: Parameters are variables in function definition; Arguments are values passed when calling

Syntax:

```
function greet(name) { // 'name' is parameter
  console.log(`Hello, ${name}`);
}
greet("Ahmed"); // 'Ahmed' is argument
```

Example with Multiple Parameters:

```
function add(a, b) {
  console.log(a + b);
}
add(5, 10); // Output: 15
```

Key Points:

- Parameters let functions work with different data
- Can have multiple parameters separated by comma
- Order of arguments matters

9. Return Statement

Definition: Sends a value back from function to where it was called

Syntax:

```
function functionName() {
  return value;
}
let result = functionName();
```

Example:

```
function add(a, b) {
  return a + b;
}
let sum = add(5, 3); // sum = 8
```

Key Points:

- Function stops executing after return
- Returned value can be stored in variable
- Very useful for calculations

10. First-Class Functions

Definition: Functions can be assigned to variables and passed as parameters

Assigning Function to Variable:

```
function add(a, b) {  
  return a + b;  
}  
  
let sum = add; // assign function to variable  
  
sum(5, 3); // call it like add()
```

Example:

```
function test(func) {  
  console.log(func(2, 4)); // call function inside  
}  
  
test(add); // pass add function as argument
```

Key Points:

- Functions are objects in JavaScript
- Can pass functions as parameters (callbacks)
- Can store functions in variables

11. Callbacks

Definition: Function passed as parameter to another function, executed later

Syntax:

```
function processData(data, callback) {  
  callback(data);  
}
```

Example:

```
const add = (a, b) => a + b;  
  
function calculator(x, y, operation) {  
  console.log(operation(x, y));  
}  
  
calculator(5, 3, add); // Output: 8
```

Key Points:

- Callback is function passed as argument
- Called inside another function
- Very useful for flexible code

12. Arrow Functions

Definition: Modern, shorter syntax for writing functions

Syntax:

```
const functionName = (params) => {  
  // code here  
};
```

Examples:

```
// Single parameter  
const greet = (name) => `Hello, ${name}`;  
  
// Multiple parameters  
const add = (a, b) => a + b;  
  
// Multiple lines  
const calculate = (x) => {  
  const result = x * 2;  
  return result;  
};
```

Key Points:

- Shorter than regular functions
- Use const for arrow functions
- No curly braces = implicit return

13. Scopes (Global & Local)

Definition: Region where variables can be accessed

Global Scope:

```
let globalVar = 10; // Accessible everywhere
```

Local Scope:

```
function test() {  
  let localVar = 5; // Only in this function  
}  
  
console.log(localVar); // Error! Not accessible
```

Key Points:

- Local scope prevents variable name conflicts
- Variables in function are deleted after execution
- Use let/const for block scope

14. Lexical Scoping

Definition: Inner functions can access outer function variables

Example:

```
let globalVar = "Global";

function outer() {
  let outerVar = "Outer";

  function inner() {
    console.log(globalVar); // Accessible
    console.log(outerVar); // Accessible
  }
  inner();
}
outer();
```

Key Points:

- Inner can access outer variables
- Creates scope chain
- Foundation for closures

15. Closures

Definition: Function that remembers variables from outer scope even after outer function ends

Example:

```
function counter() {
  let count = 0; // Private variable

  return function() {
    count++;
    return count;
  };
}

const c = counter();
c(); // 1
c(); // 2
c(); // 3
```

Key Points:

- Function remembers outer variables
- Creates data privacy (count can't be accessed directly)
- Very powerful for state management

16. Arrays

Definition: Ordered list of values

Creating Arrays:

```
let fruits = ["Apple", "Banana", "Orange"];  
let numbers = [1, 2, 3, 4, 5];  
let mixed = [1, "text", true];
```

Accessing Elements:

```
console.log(fruits[0]); // Apple (first element)  
console.log(fruits[2]); // Orange (third element)  
console.log(fruits.length); // 3 (total elements)
```

Common Array Methods:

```
fruits.push("Mango"); // Add at end  
fruits.pop(); // Remove from end  
fruits.unshift("Grape"); // Add at start
```

Key Points:

- Index starts at 0
- Arrays can store any data type
- length property gives number of elements

17. Array Methods

map() - Transform each element:

```
let numbers = [1, 2, 3];  
let doubled = numbers.map(n => n * 2);  
// doubled = [2, 4, 6]
```

filter() - Select specific elements:

```
let marks = [50, 30, 80, 90];  
let passed = marks.filter(m => m >= 50);  
// passed = [50, 80, 90]
```

reduce() - Combine into single value:

```
let sum = marks.reduce((acc, m) => acc + m, 0);  
// sum = 250
```

forEach() - Loop through each:

```
marks.forEach(mark => console.log(mark));
```

find() - Get first match:

```
let first = marks.find(m => m > 60);
```

```
// first = 80
```

18. Loops

for loop:

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

forEach() - Loop through array:

```
let names = ["Ahmed", "Umer", "Faizan"];  
names.forEach((name, index) => {  
  console.log(index, name);  
});
```

for...in loop - Loop through object properties:

```
let student = { name: "Ali", age: 20 };  
for (let key in student) {  
  console.log(key, student[key]);  
}
```

Key Points:

- for loop: precise control over iterations
- forEach: clean way to loop arrays
- for...in: loop through object properties

19. Objects

Definition: Collection of properties (key-value pairs)

Creating Objects:

```
const student = {  
  name: "Ahmed",  
  age: 20,  
  course: "MERN",  
  active: true  
};
```

Accessing Properties:

```
console.log(student.name); // Dot notation  
console.log(student["age"]); // Bracket notation
```

Modifying Properties:

```
student.age = 21; // Update  
student.city = "Lahore"; // Add new
```

Key Points:

- Properties can be any data type
- Access with dot (.) or brackets ([])
- Can add/modify/delete properties anytime

20. Destructuring

Definition: Extract properties from objects into separate variables

Basic Destructuring:

```
const { name, age } = student;  
console.log(name); // Ahmed  
console.log(age); // 20
```

Renaming:

```
const { name: studentName, age: studentAge } = student;  
console.log(studentName); // Ahmed
```

Nested Destructuring:

```
const student = {  
  profile: { name: "Ali", age: 20 }  
};  
const { profile: { name, age } } = student;
```

In Function Parameters:

```
function displayStudent({ name, age }) {  
  console.log(name, age);  
}
```

Key Points:

- Cleaner code
- Extract only needed properties
- Works with nested objects

21. Typed.js Library

Definition: JavaScript library for creating typing/animation effects

Installation:

```
<script  
src="https://cdn.jsdelivr.net/npm/typed.js@2/lib/typed.umd.js"></script>
```

Basic Usage:

```
var typed = new Typed("#element", {  
  strings: ["Hello", "World"],  
  typeSpeed: 50,  
  backSpeed: 40,  
  loop: true  
});
```

Key Options:

- strings: Array of text to type
- typeSpeed: Speed of typing (ms)
- backSpeed: Speed of deleting (ms)
- loop: Repeat animation continuously

Key Points:

- Great for creating engaging text animations
- Select HTML element by ID or class
- Can customize speed and behavior

22. DOM (Document Object Model)

Definition: Programming interface for HTML/CSS; lets JavaScript modify web pages

Selecting Elements:

```
document.getElementById("id");  
document.querySelector("selector");  
document.querySelectorAll("selector");
```

Modifying Content:

```
element.innerText = "New text";  
element.innerHTML = "<b>Bold text</b>";
```

Adding Event Listeners:

```
element.addEventListener("click", function() {  
  console.log("Clicked!");  
});
```

Changing Styles:

```
element.style.color = "red";  
element.style.backgroundColor = "blue";
```

Key Points:

- querySelector is most modern
- innerText for text, innerHTML for HTML
- Can respond to user interactions